

MultiTech Escape PUZZLE GAME - Complete Documentation

Table of Contents

- [1. Introduction](#)
 - [2. Installation](#)
 - [3. Theme System](#)
 - [4. Level Editor](#)
 - [5. Gameplay Systems](#)
 - [6. Scripts Reference](#)
 - [7. Asset Store Guidelines](#)
-

Introduction

MultiTech Escape PUZZLE GAME is a comprehensive Unity game template designed for creating arrow rope puzzle games. This template provides all the essential systems needed to build, customize, and publish your puzzle game.

Key Features

- **Complete Game Template:** Ready-to-use game systems
 - **Theme Customization:** Flexible theme management system
 - **Level Editor:** Visual level creation tools
 - **Modular Architecture:** Easy to extend and customize
 - **Asset Store Ready:** Optimized for Unity Asset Store distribution
-

Installation

Step 1: Import Package

1. Open Unity Hub
2. Create a new project or open an existing one (Unity 2021.3 LTS or later)
3. Import the MultiTech Escape PUZZLE GAME package
4. Wait for Unity to import all assets and compile scripts

Step 2: Verify Installation

1. Check that the following folders exist:

- `Assets/MultiTech Studio/Escape Game/`

2. Open the setup window: `Tools > MultiTech Studio > Escape Game Setup`

3. Verify that the Theme tab is accessible

Step 3: Initial Setup

1. The first time you open the setup window, 7 default themes will be automatically created
2. Themes are saved in: `Assets/MultiTech Studio/Escape Game/Themes/`
3. Review and customize themes as needed

Theme System

Overview

The theme system allows you to create multiple visual themes for your game. Each theme defines:

- Visual appearance (icons, colors, skybox)
- Arrow prefab to use
- Color palette options

Using the Theme Editor

Accessing the Theme Editor

1. Go to `Tools > MultiTech Studio > Escape Game Setup`
2. Click on the "Theme" tab

Theme List

The theme list displays up to 7 themes in a horizontal scrollable view:

- **Theme Slots:** Visual representation of each theme with its icon
- **Empty Slots:** Grayed-out slots for themes that haven't been created yet
- **Selected Theme:** Highlighted in blue when selected

Creating a New Theme

1. Click the "Create New Theme" button (if less than 7 themes exist)
2. A new theme will be created with default values:
 - Default Arrow Prefab: `Arrow.prefab`
 - Default Skybox: `Skybox Regular.mat`
 - Default Grid Icon: `Circle.png`
3. The new theme will be automatically selected

Editing Theme Details

When a theme is selected, the details panel on the right shows all editable properties:

Theme Identity

- **Theme Name:** Text field for the theme's display name
- **Theme Icon:** Sprite field for the theme selection icon

Arrow Settings

- **Arrow Prefab:** GameObject field for the arrow prefab to spawn
 - Click "Set Default Arrow Prefab" if not assigned
 - Default: `Assets/MultiTech Studio/Escape Game/Prefabs/Arrow.prefab`

Color Settings

- **Have Different Colors:** Toggle to enable multiple colors
- **Colors:** List of colors (shown when "Have Different Colors" is enabled)
 - Use the  button to add new colors
 - Use the  button to remove colors
 - Click the color bar to open the color picker
 - Drag to reorder colors

Visual Settings

- **Skybox Material:** Material field for the skybox
 - Click "Set Default Skybox" if not assigned
 - Default: `Assets/MultiTech Studio/Escape Game/Materials/Skybox Regular.mat`
- **Grid Icon:** Sprite field for the grid dot icon
 - Click "Set Default Grid Icon" if not assigned
 - Default: `Assets/MultiTech Studio/Escape Game/Sprites/Circle.png`
- **Background:** Sprite field for the background sprite
 - Optional field for custom background images
 - > **Note:** If a selected sprite appears too small or too big when applied via the editor window, adjust its **Pixels Per Unit** setting in the sprite's Import Settings. Select the sprite in the Project window, then in the Inspector, adjust the "Pixels Per Unit" value (typically between 100-1000 depending on your sprite resolution).

Applying a Theme

After configuring a theme, click the "APPLY" button to apply all theme settings to the current scene:

1. **Arrow Prefab:** Sets the arrow prefab in LevelManager
2. **LineColors:** Configures the LineColors component on the arrow prefab (enables/disables and copies colors)
3. **Skybox Material:** Applies the skybox material to the scene

4. **Background Sprite:** Sets the background sprite on the Background GameObject
5. **Grid Icon:** Applies the grid icon to the Dot prefab and rebuilds the grid
6. **Theme Icon:** Sets the theme icon on the Icon Image component in the UI
7. **CurrentTheme Asset:** Automatically creates or updates the CurrentTheme ScriptableObject for runtime theme access

> **Note:** When you apply a theme in the editor, a `CurrentTheme.asset` file is automatically created in the Themes folder. This asset holds the reference to the currently active theme and can be used by runtime scripts for theme switching.

Deleting a Theme

1. Select the theme to delete
2. Click the "Delete Theme" button at the bottom of the details panel
3. Confirm the deletion in the dialog

Theme ScriptableObject

Themes are stored as ScriptableObjects of type `ThemeData` :

```
using UnityEngine;

[CreateAssetMenu(fileName = "ThemeData", menuName = "Escape Game/Theme Data")]
```

```
public class ThemeData : ScriptableObject
```

```
{
```

```
    public string themeName;
```

```
    public Sprite themeIcon;
```

```
    public GameObject arrowPrefab;
```

```
    public bool haveDifferentColors;
```

```
    public List<Color> colors;
```

```
    public Material skyboxMaterial;
```

```
    public Sprite gridIcon;
```

```
}
```

You can also create themes manually:

1. Right-click in the Project window
2. Select `Create > Escape Game > Theme Data`
3. Configure the theme properties in the Inspector

Default Themes

On first launch, 7 default themes are created based on existing prefabs:

1. **Arrow Neon**: Based on Arrow Neon prefab with colorful palette
2. **Snake 1**: Based on Snake 1 prefab
3. **Snake 2**: Based on Snake 2 prefab
4. **Snake 3**: Based on Snake 3 prefab
5. **Train**: Based on Train prefab
6. **Train 1**: Based on Train 1 prefab
7. **Arrow**: Based on Arrow prefab

Colors are automatically extracted from the `LineColors` component on each prefab.

Runtime Theme System

The theme system supports runtime theme switching through the `CurrentTheme` ScriptableObject and `ThemeManager` component.

CurrentTheme ScriptableObject

`CurrentTheme` is a ScriptableObject that holds the reference to the currently active theme. It's automatically created when you apply a theme in the editor. **Location:**

`Assets/MultiTech Studio/Escape Game/Themes/CurrentTheme.asset`

Properties:

- `ActiveTheme` (ThemeData): The currently active theme
- `OnThemeChanged` (Action): Event triggered when the active theme changes

Usage:

```
// Get the active theme CurrentTheme currentTheme = // Load from Resources or assign in Inspector
```

```
ThemeData activeTheme = currentTheme.ActiveTheme;
```

```
// Change theme at runtime currentTheme.ActiveTheme = newTheme; // This triggers OnThemeChanged event
```



ThemeManager Component

ThemeManager is a MonoBehaviour that applies themes at runtime. Add it to a GameObject in your scene to enable runtime theme switching. **Setup:**

1. Create an empty GameObject (e.g., "ThemeManager")
2. Add the **ThemeManager** component
3. Assign the **CurrentTheme** asset (created automatically when applying themes)
4. Assign scene references:
 - **LevelManager**: For applying arrow prefab
 - **GridManager**: For applying grid icon
 - **Background Renderer**: SpriteRenderer for background sprite
 - **Theme Icon Image**: UI Image component for theme icon
5. Enable **Apply On Start** to automatically apply the theme when the scene loads

Features:

- Automatically applies theme on Start (if enabled)
- Listens to **CurrentTheme.OnThemeChanged** event for runtime theme switching
- Updates all visual elements (skybox, background, grid, arrows, icons)
- Can be accessed via **ThemeManager.I** static reference

Runtime Theme Switching:

```
// Get ThemeManager instance
```

```
ThemeManager themeManager = ThemeManager.I;
```

```
// Change theme
```

CurrentTheme currentTheme = // Your CurrentTheme asset reference

currentTheme.ActiveTheme = newThemeData; // ThemeManager will automatically apply it

LevelManager Theme Integration

`LevelManager` can optionally use the `CurrentTheme` asset to automatically apply the theme's arrow prefab at runtime. **Setup:**

1. Select the GameObject with `LevelManager` component
2. In the Inspector, find the **Theme (Optional)** section
3. Assign the `CurrentTheme` asset
4. At runtime, `LevelManager` will automatically use the arrow prefab from the active theme

Note: If `CurrentTheme` is not assigned, `LevelManager` will use the `arrowLinePrefab` field directly (backward compatible).

Level Editor

Overview

The level editor allows you to create and edit game levels visually using a runtime painting system. You can draw arrow paths directly in the Game view during Play mode.

Creating a Level

Follow these steps to create a new level:

Step 1: Open the Level Editor Scene

1. Open the scene: `Assets/MultiTech Studio/Escape Game/Scenes/LevelEditor.unity`

Step 2: Verify Scene Setup

Ensure the scene has all required components and GameObjects as shown in the Hierarchy:

- **Main Camera:** Camera for the game view
- **Directional Light:** Scene lighting

- **Global Volume:** Post-processing volume (optional)
- **Grid:** Parent GameObject containing the grid system
- **LevelMaanger:** Level management component
- **RuntimePainter:** Runtime arrow painting component
- **GameMode:** Game mode toggle component

Step 3: Configure GameMode Component

1. Select the **GameMode** GameObject in the Hierarchy

2. In the Inspector, set:

- **Mode:** `Draw` (required for painting)
- **Show Hud:** Optional (toggles the HUD display)

Step 4: Configure RuntimePainter Component

1. Select the **RuntimePainter** GameObject in the Hierarchy

2. In the Inspector, assign all required components:

- **Grid:** Assign the Grid GameObject (GridManager component)
- **Arrow Line Prefab:** Assign your arrow prefab (e.g., `Arrow.prefab`)
- **Preview Line:** Assign the LineRenderer component on RuntimePainter
- **Game Camera:** Assign the Main Camera

3. Set the **Level Asset Name** field with your desired level name (e.g., "New Level")

- This name will be used when saving the level asset

Step 5: Configure LevelManager Component

1. Select the **LevelManager** GameObject in the Hierarchy

2. In the Inspector:

- **Arrow Line Prefab:** Assign the arrow prefab (same as RuntimePainter)
- **Levels:** Must be set to `0` (empty list) - otherwise it will spawn that level instead of allowing you to paint
- Other fields can be left unassigned for level creation

Step 6: Configure GridManager Component

1. Select the **Grid** GameObject in the Hierarchy

2. In the Inspector:

- **Build in Editor**: Must be checked ✓
- **Dot Prefab**: Must be assigned
- **Dots Parent**: Must be assigned

3. Configure grid dimensions:

- **Width**: Number of columns
- **Height**: Number of rows
- **Spacing**: Distance between dots
- **Origin**: Starting position of the grid

Step 7: Start Painting

1. Press **Play** to enter Play mode

2. In the Game view, click and drag on dots to paint arrow paths:

- Click on a dot to start a path
- Drag across neighboring dots to extend the path
- Release to complete the arrow path

3. Use the GUI buttons in the top-left corner:

- ↶ **Undo**: Remove the last painted arrow
-  **Save Arrows as Level Asset**: Save the current level

Step 8: Adjust Grid During Play (Optional)

If you need to change the grid size while painting:

1. While in Play mode, select the **Grid** GameObject

2. In the Inspector, modify:

- **Width**: Number of columns

- **Height:** Number of rows

- **Spacing:** Distance between dots

- **Origin:** Grid starting position

3. The grid will update immediately in the Game view

Step 9: Set Camera Zoom (Optional)

To set the initial camera zoom for the level:

1. While in Play mode, select the **Main Camera** GameObject

2. In the Inspector, adjust the **Orthographic Size** value

3. This camera size will be saved in the LevelAsset when you save the level

Step 10: Save the Level

1. Click the  **Save Arrows as Level Asset** button in the top-left GUI

2. The level will be saved to: `Assets/MultiTech Studio/Escape Game/Levels/`

3. The filename will be based on the **Level Asset Name** you set in RuntimePainter

Step 11: Test the Level

1. **Stop Play mode**

2. Select the **LevelMaanger** GameObject

3. In the Inspector, add your saved level to the **Levels** list:

- Click the  button to add an element

- Drag your saved level asset from the Project window to the list

4. Press Play again to test the level

Step 12: Mode Toggle

During Play mode, you can toggle between drawing and testing:

- **Press ** : Toggle between `Draw` and `Play` modes

- **Draw Mode:** Allows you to paint arrow paths

- **Play Mode:** Disables painting, allows arrow movement

- **Press SPACE** (in Play mode): Move all arrows one step forward to test the level

> **Important:** After moving arrows with SPACE in Play mode, you need to clean all arrows from the scene before saving the LevelAsset. Arrows that have moved may not be in their original grid positions, which can cause issues. Switch back to Draw mode to start drawing arrows again.

Level Asset Structure

Levels are stored as ScriptableObjects:

```
csharp [CreateAssetMenu(fileName = "LevelAsset", menuName = "Levels/Level Asset")]
```

```
public class LevelAsset : ScriptableObject
```

```
{
```

```
    public string levelName;
```

```
    public GridDef grid;
```

```
    public List arrows;
```

```
    public float camSize;
```

```
    public int coinsEarned;
```

```
    public LevelDifficulty levelDifficulty;
```

```
}
```

Level Properties

- **Level Name:** Display name for the level
 - **Grid:** Grid dimensions and spacing
 - **Arrows:** List of arrow paths and configurations
 - **Camera Size:** Camera orthographic size for the level
 - **Coins Earned:** Coins awarded for completing the level
 - **Difficulty:** Easy, Medium, Hard, or VeryHard
-

Gameplay Systems

Arrow System

The `ArrowLine` component handles arrow movement and behavior:

- **Path Following:** Arrows follow a predefined path through grid dots
- **Collision Detection:** Arrows detect collisions with other arrows
- **Exit Detection:** Arrows detect when they leave the grid
- **Animation:** Smooth movement using DOTween

Grid System

The `GridManager` manages the game grid:

- **Grid Generation:** Creates a grid of dots based on configuration
- **Dot Management:** Tracks dot occupancy and states
- **Coordinate System:** Converts between grid and world coordinates

Level Management

The `LevelManager` handles level progression:

- **Level Loading:** Loads levels from LevelAsset ScriptableObjects
- **State Management:** Tracks level state (NotStarted, Playing, Completed, Failed)
- **Arrow Spawning:** Instantiates arrows based on level data
- **Win/Lose Conditions:** Determines level completion

UI System

The `UiManager` manages all UI elements:

- **Menu Navigation:** Main menu, level selection, gameplay UI
 - **Level Display:** Shows current level number
 - **Coin Display:** Shows player coins
 - **Lives Display:** Shows remaining lives
 - **Win/Lose Screens:** Displays completion screens
-

Scripts Reference

Core Scripts

ThemeData.cs

ScriptableObject for storing theme configuration. **Properties:**

- `themeName` (string): Theme display name
- `themeIcon` (Sprite): Icon for theme selection
- `arrowPrefab` (GameObject): Arrow prefab to use
- `haveDifferentColors` (bool): Enable multiple colors
- `colors` (List): Color palette
- `skyboxMaterial` (Material): Skybox material
- `gridIcon` (Sprite): Grid dot sprite
- `background` (Sprite): Background sprite

CurrentTheme.cs

ScriptableObject that holds the reference to the currently active theme for runtime access.

Properties:

- `ActiveTheme` (ThemeData): The currently active theme (setting this triggers `OnThemeChanged`)
- `OnThemeChanged` (Action): Event triggered when the active theme changes

Usage:

- Automatically created when applying themes in the editor
- Can be referenced by runtime scripts for theme switching
- Provides a single source of truth for the active theme

ThemeManager.cs

MonoBehaviour component for runtime theme application and management. **Properties:**

- `currentThemeAsset` (CurrentTheme): Reference to the CurrentTheme ScriptableObject
- `levelManager` (LevelManager): Reference to LevelManager for arrow prefab
- `gridManager` (GridManager): Reference to GridManager for grid icon
- `backgroundRenderer` (SpriteRenderer): Reference to background sprite renderer
- `themeIconImage` (Image): Reference to theme icon UI Image

- `applyOnStart` (bool): Automatically apply theme on Start

Key Methods:

- `ApplyTheme(ThemeData)` : Applies the given theme to all scene objects
- `SetActiveTheme(ThemeData)` : Sets the active theme in CurrentTheme asset
- `GetActiveTheme()` : Gets the currently active theme

Static Access:

- `ThemeManager.I` : Static reference to the ThemeManager instance

ArrowLine.cs

Component for arrow movement and behavior. **Key Methods:**

- `TryAdvance()` : Moves arrow forward one step
- `SetBaseColor(Color)` : Sets the arrow's base color
- `SyncVisualImmediate()` : Updates visual representation

GridManager.cs

Manages the game grid system. **Key Methods:**

- `GetDotByGrid(Vector2Int)` : Gets dot at grid coordinates
- `ToGridDef()` : Converts current grid to GridDef

LevelManager.cs

Handles level loading and management. **Key Methods:**

- `LoadLevel(int)` : Loads a level by index
- `ReloadLevel()` : Reloads the current level
- `ApplyThemeArrowPrefab()` : Applies arrow prefab from CurrentTheme (called automatically on Start)

Theme Integration:

- Optional `currentThemeAsset` field: If assigned, automatically uses arrow prefab from active theme at runtime
- Backward compatible: Works without CurrentTheme assigned (uses `arrowLinePrefab` field directly)

UiManager.cs

Manages UI elements and navigation. **Key Methods:**

- `LoadLevel(int)` : Loads level and updates UI
- `UpdateLevelText()` : Updates level display
- `UpdateCoins(int)` : Updates coin display

AdsManager.cs

Manages AdMob advertisements (conditionally compiled with `MT_ADMOB`). **Key Methods:**

- `LoadBannerAd()` : Loads banner advertisement
- `ShowInterstitialAd()` : Shows interstitial advertisement
- `ShowRewardedAd(System.Action)` : Shows rewarded video advertisement
- `CanShowInterstitialAfterLevel(int)` : Checks if interstitial can be shown after level
- `IsInterstitialReady()` : Checks if interstitial ad is ready
- `IsRewardVideoReady()` : Checks if rewarded video is ready

Properties:

- `bannerId` (string): Banner ad unit ID
- `interId` (string): Interstitial ad unit ID
- `rewardedInterId` (string): Rewarded interstitial ad unit ID
- `rewardedId` (string): Rewarded video ad unit ID
- `interstitialLevelThreshold` (int): Minimum level for interstitial ads
- `canShowInterstitialAfterLevel` (bool): Enable/disable interstitial ads

FirebaseManager.cs

Manages Firebase initialization and analytics (conditionally compiled with `MT_FIREBASE`).

Key Methods:

- `SendLevelData()` : Sends level completion data to Firebase Crashlytics
- `Initialize()` : Initializes Firebase SDK

AutoDefines.cs

Editor script that automatically manages scripting define symbols based on SDK detection.

Key Methods:

- `UpdateDefines()` : Automatically called on project load to detect SDKs
- `ForceUpdateDefines()` : Manually refresh SDK detection
- `ClearAllSDKDefines()` : Remove all SDK defines for testing

Editor Scripts

EscapeGameSetupWindow.cs

Main editor window for theme management and SDK configuration. **Access:** `Tools >`

`MultiTech Studio > Escape Game Setup` **Features:**

- Theme creation and editing
- Default theme generation
- Asset path management
- SDK status monitoring
- SDK enable/disable toggles
- AdMob configuration
- Compilation testing

LevelSaveTool.cs

Tool for saving levels from scenes. **Access:** `Tools > Arrows > Save Level As New`

`Asset` or `Ctrl+Alt+S` **Features:**

- Extracts level data from scene
- Creates LevelAsset ScriptableObject
- Saves to Levels folder

Asset Store Guidelines

Package Structure

For Asset Store distribution, ensure:

1. **Clean Project Structure:** All assets organized in `Assets/MultiTech Studio/Escape Game/`
2. **Documentation:** Include README.md and Document.md
3. **Example Scenes:** Provide example scenes demonstrating features
4. **License Files:** Include all required license files in Documents folder

Required Files

- `README.md` : Quick start guide
- `Document.md` : Complete documentation (this file)

- `LICENSE.txt` : License information
- Example scenes with sample levels
- Demo/preview materials

Best Practices

1. **Code Organization:** Keep scripts organized by functionality 2. **Naming Conventions:** Use clear, descriptive names 3. **Comments:** Document complex logic 4. **Error Handling:** Include proper error handling and validation 5. **Performance:** Optimize for performance (object pooling, etc.)

Testing Checklist

Before publishing:

- ☐ All scripts compile without errors
- ☐ Theme editor opens and functions correctly
- ☐ Default themes are created on first launch
- ☐ Level editor saves levels correctly
- ☐ Example scenes run without errors
- ☐ UI navigation works correctly
- ☐ All default assets are assigned
- ☐ Documentation is complete and accurate

SDK Integration (AdMob & Firebase)

Overview

The MultiTech Escape PUZZLE GAME template supports optional integration with Google Mobile Ads (AdMob) and Firebase. The SDK system uses conditional compilation to ensure the project compiles safely with or without these SDKs installed.

Accessing SDK Settings

1. Open the setup window: `Tools > MultiTech Studio > Escape Game Setup` 2. Click on the "Settings" tab 3. View SDK status and manage SDK integration

SDK Status

The Settings tab displays the current status of each SDK:

- **✓ Available:** SDK is detected and available in the project
- **✗ Not Available:** SDK is not installed or not detected

Enabling/Disabling SDKs

You can enable or disable SDKs using checkboxes in the Settings tab:

- **Enable Google Mobile Ads (AdMob):** Toggles AdMob integration
- **Enable Firebase:** Toggles Firebase integration

When enabled, the corresponding scripting define symbol is added:

- `MT_ADMOB` for AdMob
- `MT_FIREBASE` for Firebase

When disabled, the define is removed and all SDK-specific code is excluded from compilation.

AdMob Configuration

When AdMob is enabled, the Settings tab displays AdMob configuration fields:

Prerequisites

1. Add `AdsManager` component to a GameObject in your scene 2. The component must be present for configuration to appear

Configuration Fields

Ad Unit IDs:

- **Banner ID:** Ad unit ID for banner advertisements
- **Interstitial ID:** Ad unit ID for interstitial advertisements
- **Rewarded Interstitial ID:** Ad unit ID for rewarded interstitial advertisements
- **Rewarded ID:** Ad unit ID for rewarded video advertisements

Interstitial Settings:

- **Interstitial Level Threshold:** Minimum level number before interstitial ads can be shown
- **Can Show Interstitial After Level:** Toggle to enable/disable interstitial ads after the threshold level

Default Values

The AdsManager comes with test ad unit IDs by default:

- Banner: `ca-app-pub-3940256099942544/6300978111`
- Interstitial: `ca-app-pub-3940256099942544/1033173712`
- Rewarded Interstitial: `ca-app-pub-3940256099942544/8691691433`
- Rewarded: `ca-app-pub-3940256099942544/5224354917`

> **Note:** Replace test IDs with your actual AdMob ad unit IDs before publishing.

Firebase Setup

1. Add `FirebaseManager` component to a GameObject in your scene 2. Configure Firebase in your Firebase console 3. Download and import Firebase configuration files (google-services.json for Android, GoogleService-Info.plist for iOS) 4. The system automatically initializes Firebase when enabled

Conditional Compilation

The SDK system uses conditional compilation directives (`#if MT_ADMOB` , `#if MT_FIREBASE`) to include or exclude SDK-specific code:

- **AdsManager.cs:** All AdMob API calls are wrapped in `#if MT_ADMOB` blocks
- **FirebaseManager.cs:** All Firebase API calls are wrapped in `#if MT_FIREBASE` blocks

This ensures:

- The project compiles without SDKs installed
- No runtime errors when SDKs are disabled
- Easy toggling between SDK-enabled and SDK-disabled builds

AutoDefines System

The `AutoDefines.cs` editor script automatically manages scripting define symbols:

- Detects SDK presence on project load
- Adds defines when SDKs are detected
- Removes defines when SDKs are not found
- Runs automatically when Unity loads the project

Settings Tab Actions

Update Defines:

- Manually refresh SDK detection
- Updates scripting define symbols based on current SDK availability

Clear All SDK Defines:

- Removes all SDK defines (`MT_ADMOB` , `MT_FIREBASE`)
- Useful for testing compilation without external dependencies
- Shows confirmation dialog before clearing

Test Compilation:

- Verifies that the project compiles correctly
- Shows compilation status in the console

Refresh Project:

- Refreshes Unity's asset database
- Useful after importing SDKs or making changes

Best Practices

1. **Development:** Keep SDKs disabled during development if not needed 2. **Testing:** Use test ad unit IDs during development 3. **Production:** Replace test IDs with real AdMob ad unit IDs before publishing 4. **Version Control:** SDK defines are stored in ProjectSettings, commit these changes 5. **Team Collaboration:** Ensure all team members have the same SDK setup

Troubleshooting SDK Issues

Problem: SDK not detected

- **Solution:** Verify SDK is properly imported
- **Solution:** Check SDK assemblies are referenced
- **Solution:** Click "Update Defines" to refresh detection

Problem: AdsManager not found in scene

- **Solution:** Add `AdsManager` component to a GameObject in the active scene

Problem: Compilation errors with SDK enabled

- **Solution:** Verify SDK is properly installed
- **Solution:** Check SDK version compatibility
- **Solution:** Use "Test Compilation" to identify issues
- **Solution:** Temporarily disable SDK if needed

Problem: Ads not showing

- **Solution:** Verify AdMob is enabled in Settings tab
 - **Solution:** Check ad unit IDs are correct
 - **Solution:** Ensure AdsManager component is in the scene
 - **Solution:** Verify internet connection for ad requests
-

Troubleshooting

Theme Editor Issues

Problem: Themes not appearing

- **Solution:** Check that Themes folder exists and contains .asset files
- **Solution:** Refresh the Asset Database (Ctrl+R)

Problem: Default assets not found

- **Solution:** Verify asset paths in EscapeGameSetupWindow.cs match your project structure
- **Solution:** Manually assign assets using the "Set Default" buttons

Problem: Sprites appear too small or too big after applying theme

- **Solution:** Adjust the sprite's **Pixels Per Unit** setting in Import Settings. Select the sprite in the Project window, then in the Inspector, adjust the "Pixels Per Unit" value (typically between 100-1000 depending on your sprite resolution)

Problem: ThemeManager not applying theme at runtime

- **Solution:** Ensure CurrentTheme asset is assigned to ThemeManager component
- **Solution:** Check that all scene references are assigned (LevelManager, GridManager, Background Renderer, Theme Icon Image)
- **Solution:** Verify "Apply On Start" is enabled if you want automatic theme application on scene load
- **Solution:** Check Unity console for error messages

Problem: CurrentTheme asset not found

- **Solution:** Apply a theme in the Theme Editor - this automatically creates CurrentTheme.asset
- **Solution:** Or manually create: Right-click in Project window > Create > Tools/MultiTech Studio/Escape Game Current Theme

Level Editor Issues

Problem: Levels not saving

- **Solution:** Ensure Levels folder exists
- **Solution:** Check that GridManager and ArrowLine components are properly configured
- Problem:** Arrows not moving
- **Solution:** Verify ArrowLine component has nodes assigned
- **Solution:** Check that DOTween is imported and configured

SDK Integration Issues

Problem: SDK not detected

- **Solution:** Verify SDK is properly imported into the project
- **Solution:** Check that SDK assemblies are referenced
- **Solution:** Click "Update Defines" in Settings tab to refresh detection
- **Solution:** Verify SDK installation in Package Manager or Asset Store

Problem: AdsManager not found in scene

- **Solution:** Add `AdsManager` component to a GameObject in your scene
- **Solution:** Ensure the component is in the active scene

Problem: Compilation errors with SDK enabled

- **Solution:** Ensure SDK is properly installed and imported
- **Solution:** Check SDK version compatibility with your Unity version
- **Solution:** Try "Test Compilation" button in Settings tab
- **Solution:** If issues persist, disable the SDK temporarily using the toggle

Problem: Ads not showing

- **Solution:** Verify AdMob is enabled in Settings tab
- **Solution:** Check ad unit IDs are correct (not using test IDs in production)
- **Solution:** Ensure AdsManager component is in the scene and initialized
- **Solution:** Verify internet connection for ad requests
- **Solution:** Check AdMob console for ad unit status

General Issues

Problem: Scripts not compiling

- **Solution:** Ensure all required dependencies are imported
 - **Solution:** Check Unity version compatibility (2021.3 LTS or later)
 - **Solution:** Verify SDK defines are set correctly if using AdMob/Firebase
-

Support and Updates

For support, feature requests, or bug reports, please refer to the project repository or contact the developer.

Version History

Version 1.0.0

- Initial release
 - Theme system with editor window
 - Level editor with save functionality
 - Complete gameplay systems
 - UI management system
 - Documentation
-

Last Updated: [Current Date]